like you to replace the right hand of the initial assignment to the attribute or constant with a call to the Sorbet method T.let. You can see an example in the intialize method: @signatures = T.let({}, T::Hash[String, T::Array[String]]). The T.let method takes two arguments: the first is the actual value being assigned, and the second is the type of the variable going forward. In this case, the assignment to an empty hash is not enough information to tell Sorbet that the hash actually has String keys and Array of String values.

Sorbet treats attr_reader and attr_writer like any other method, so they need to have sig annotations. For attr_accessor, which defines both a reader and a writer, the sig annotation should be for the reader, and Sorbet will infer the writer.

The type value is usually just the Ruby type name, with a couple of exceptions. As you can see in this file, Arrays and Hashes are handled specially, through types defined in Sorbet's T module. Arrays are specified as T::Array[TYPE] and hashes are T::Hash[KEY TYPE, VALUE TYPE]. There's also a special T::Boolean for true or false values.

If you want to specify that a value can optionally be nil, as with the return value to lookup, then you wrap the value in a call to T.nilable. A union type is specified with T.any as in T.any(String, Integer).

There's more to the Sorbet type system; full documentation can be found at https://sorbet.org/docs.

Using Sorbet

Sorbet allows for static type checking and runtime type checking. The static checking comes from a command line:

```
$ bundle exec srb tc FILES
```

Usually you'll want to run your entire project, which you can do like so:

```
$ srb tc .
No errors! Great job.
```

On the first pass, the T.nilable was missing, and Sorbet definitely noticed—the output included errors like: aaagmnr/lib/aaagmnr/finder.rb:25: Expected T::Array[String] but found T.nilable(T::Array[String]) and aaagmnr/lib/aaagmnr/runner.rb:20: This code is unreachable.

Alternately you can list one or more specific paths (.rb or .rbi files after the tc) to limit checking, or you can use the --ignore=PATTERN flag to take specific files out. The --autocorrect option will give you a limited amount of autocorrect.

Sorbet will also check code at runtime, which we can verify by going into the gem's console and trying to pass in a non-array of strings to the runner:

```
$ bin/console
irb(main):001:0> Aaagmnr::Runner.new(3)
/Users/noel/.rbenv/versions/3.1.2/lib/ruby/gems/3.1.0/gems/sorbet-runtime-0.5.10346/
lib/types/configuration.rb:296:in `call_validation_error_handler_default': \
Parameter 'argv':
Expected type T::Array[String], got type Integer with value 3 (TypeError)
```

Here Sorbet is telling us that if we pass the Runner a 3 instead of an array of words, that violates the type signature of the runner. The advantage here, at least in theory, is that the error is detected as soon as possible, even before some message is called that would trigger a different